

# DSA Activity 1: Binary Search Tree (BST) Implementation

Platform: Falsicode Code Audit System | Language: Python 3.x (.py) | Score: 100 Pts

## 1. Assignment Overview

In this activity, you will implement a fully functional **Binary Search Tree (BST)** from scratch in Python. A Binary Search Tree is a hierarchical data structure where each node contains a key and up to two child subtrees. For every node, all values in the left subtree must be strictly less than the node's key, and all values in the right subtree must be greater.

### Key Learning Objectives:

- Understand and implement recursive node-based data structures.
- Master recursive and iterative tree traversals and logarithmic searching techniques.
- Apply optimal time and space complexity practices in algorithm design.
- Prepare standard, syntax-clean Python source code for automated AST and structural analysis.

## 2. Technical Specifications & Requirements

Your Python file must define two core classes: `Node` and `BinarySearchTree` (or `BST`).

| Component                  | Specification & Description                                                                                                                                       | Complexity                                        |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| <b>Class: Node</b>         | Represents an individual node in the BST.<br>• <code>__init__(self, key)</code> : Stores integer key, sets <code>left = None</code> , <code>right = None</code> . | $O(1)$ Space                                      |
| <b>insert(key)</b>         | Inserts a new key into the BST while strictly preserving the BST invariant. Duplicate keys can be ignored or handled consistently.                                | Time: $O(h)$<br>Avg: $O(\log N)$<br>Worst: $O(N)$ |
| <b>search(key)</b>         | Searches for the key in the tree. Returns <code>True</code> if found, otherwise returns <code>False</code> .                                                      | Time: $O(h)$<br>Avg: $O(\log N)$<br>Worst: $O(N)$ |
| <b>inorder_traversal()</b> | Performs an in-order depth-first traversal (Left $\rightarrow$ Root $\rightarrow$ Right) and returns a Python list of all keys in ascending order.                | Time: $O(N)$<br>Space: $O(N)$                     |
| <b>find_min()</b>          | Traverses to the leftmost node and returns the smallest key in the BST. Returns <code>None</code> if the tree is empty.                                           | Time: $O(h)$<br>Space: $O(1)$                     |
| <b>find_max()</b>          | Traverses to the rightmost node and returns the largest key in the BST. Returns <code>None</code> if the tree is empty.                                           | Time: $O(h)$<br>Space: $O(1)$                     |

## 3. Example Test Driver & Expected Output

Your code should be verifiable when tested with standard driver code such as the following:

```
# Sample Verification Driver:
bst = BinarySearchTree()
keys_to_insert = [50, 30, 70, 20, 40, 60, 80]
for k in keys_to_insert:
    bst.insert(k)

print('Inorder:', bst.inorder_traversal()) # Expected: [20, 30, 40, 50, 60, 70, 80]
print('Search 40:', bst.search(40))       # Expected: True
print('Search 99:', bst.search(99))       # Expected: False
print('Minimum:', bst.find_min())         # Expected: 20
print('Maximum:', bst.find_max())         # Expected: 80
```

4. Submission Guidelines & Constraints

- **File Format:** Submit your solution as a single Python source file (e.g., bst\_solution.py).
- **No External Tree Libraries:** Do not import external tree packages. Use standard Python only.
- **Code Cleanliness & Documentation:** Include comments stating time complexity for methods.
- **Syntax Validity:** Ensure your code compiles and runs cleanly without syntax errors before uploading.

5. Grading Rubric (100 Points Total)

| Criteria                  | Description                                                                      | Points  |
|---------------------------|----------------------------------------------------------------------------------|---------|
| Core BST Operations       | Accurate implementation of insert, search, find_min, and find_max.               | 40 Pts  |
| In-Order Traversal        | Correct traversal producing an ascending sorted sequence of keys in O(N) time.   | 25 Pts  |
| Edge Case Handling        | Handles empty trees, single-node trees, non-existent search keys gracefully.     | 15 Pts  |
| Code Quality & Complexity | Clean structure, readable variable names, and accurate complexity documentation. | 20 Pts  |
| Total Score               | Full marks achievable for complete, tested implementation.                       | 100 Pts |

Academic Integrity & Research Notice:

This activity is part of the *Falsicode: Code Plagiarism Detection in DSA using AST, N-Grams, and TF-IDF* research study. Submissions will be evaluated using Abstract Syntax Tree (AST) tokenization, normalized N-Gram matching, and TF-IDF similarity vectors. Please ensure you write your own original code.